

Review Report: origin-overhaul-phased Latest Push

Trace facts, live LSP integration, gas/static opcode demo, and next phased implementation plan

Prepared: 2026-05-24

Reviewed branch: micahscopes/fe origin-overhaul-phased

Review mode: static GitHub review; build/test execution not independently verified in this sandbox

Executive verdict: This latest push is a meaningful improvement. It moves the branch from a fixture-only trace UX prototype toward a real trace pipeline: JSONL bundles with metadata, a small typed TraceFact model, real compiler-emitted MIR facts, actual EVM bytecode instruction/opcode/gas facts, a read-only query service, LSP-side introspection hooks, configurable inlays, and a CLI path that can discover a running LSP server. It is not done: live HTTP trace queries still return an unavailable stub, compiler-emitted traces do not yet carry enough loop/source/storage/zext causality to answer Sean-style questions end-to-end, and the static gas demo must remain clearly labeled as a static opcode inventory rather than runtime gas tracing.

1. Scope and review caveats
 2. What improved in the latest push
 3. Current capability assessment
 4. Critical findings and risks
 5. Module-by-module review
 6. Sean/Fibonacci probing status
 7. LSP/CLI live connection assessment
 8. Gas-cost demo assessment
 9. SSOT and schema guardrails
 10. Recommended next phased push
 11. Acceptance criteria
 12. Implementor directive
- Appendix A. Reviewed source map

1. Scope and review caveats

Reviewed branch: micahscopes/fe, branch origin-overhaul-phased. The review focused on trace architecture, fixture/real-trace separation, trace-facts, trace-query, MIR/codegen trace emission, LSP live server scaffolding, inlay hints, configuration, and gas/static opcode reporting.

Verification caveat: I reviewed source via GitHub and did not independently run cargo, trybuild, the CLI, or the language server in this sandbox. Treat this as a static architecture and code-shape review, not a build/test certification.

Architectural baseline: The desired architecture remains phase-owned identity and phase-owned emitted facts, followed by downstream read-only query/report surfaces. CLI, LSP, HTTP, browser, graph, Datalog, and agent tooling should all consume the same trace snapshot rather than inventing parallel truth.

2. What improved in the latest push

- **README honesty improved:** The branch README now explicitly says the Fibonacci UX prototype under fe dev trace-fixture is fixture-backed and that real trace emission currently contains phase-owned MIR facts and actual EVM bytecode instruction facts, while loop membership, source-local labels, storage allocation, and zext causality are still gaps. That wording is good and should be preserved.
- **Real trace emission exists:** fe dev trace emit now has a real-trace path that builds a runtime package, emits MIR-owned facts, emits Sonatina/EVM bytecode, and emits bytecode instruction/opcode/static-gas facts. This is a real step beyond the previous hard-coded fixture-only diagnostic.
- **Trace JSONL is now a boundary artifact:** trace-facts has TraceBundle, TraceMetadata, a schema version, metadata/fact JSONL records, exactly-one-metadata read behavior, fixture-vs-compiler-emitted data-source markers, and snapshot validation. That is the right shape for CLI/LSP/browser/agent consumers.
- **TraceFact model gained useful but bounded surface:** TraceFact now includes OriginNode, OriginEdge, CompilerEvent, Storage, Instruction, InstructionCategory, LoopMembership, InlineContext, Opcode, and GasCost. That is broad enough for Sean-style probing but still much healthier than the old relation-system sprawl.
- **Trace-query is read-only and downstream:** TraceIntrospectionService consumes a validated TraceSnapshot and produces reports. This preserves the correct direction: compiler phases emit facts; query/report layers derive answers.
- **LSP integration is forming:** The language server has a service_for_file path that can build a TraceIntrospectionService from the current database, and inlay hints can use that service for configurable loop/gas hints. This is the right integration point.
- **CLI-to-LSP discovery is sketched:** fe dev trace live reads .fe-lsp.json and posts a JSON query to the LSP HTTP trace endpoint. The endpoint is currently stubbed, but the discovery flow is in the right architectural basin.

3. Current capability assessment

Capability	Current status	Assessment	Next move
Typed origin identity	Present in common and phase crates	Good foundation	Keep stable; do not centralize semantic origin meaning in common
Trace facts	Small typed model with 10 variants including opcode/gas	Good direction	Harden serde validation and remove/validate duplicate kind fields
Trace JSONL	Bundle metadata + fact records, schema version, data-source marker	Good boundary	Enforce schema compatibility and keep metadata required
Fixture reports	Still available under trace-fixture	Useful UX demo	Keep explicitly fixture-backed
Real trace emit	MIR facts + actual EVM bytecode instruction/opcode/static gas facts	Meaningful progress	Add source/local labels, loop facts, origin edges, storage reasons, zext events
Loop-cost over real trace	Reports unavailable when loop membership is absent; can show bytecode summary	Honest	Do not market as real loop attribution yet
Explain-local over real trace	Limited by lack of source-local labels and causal edges	Early	Add Symbol/Display facts and local mapping
Gas breakdown	Static opcode-table estimate from emitted bytecode facts	Good demo if labeled carefully	Clarify static/non-runtime nature and dynamic opcode uncertainty
LSP inlays	Config-gated gas/loop hints can call <code>service_for_file</code>	Promising but demo-level	Cache service results and anchor hints to real spans
Live CLI to LSP	CLI posts to <code>/trace/query</code> ; server endpoint returns unavailable stub	Scaffold only	Wire endpoint to backend actor and introspection service

4. Critical findings and risks

- **[High] Live CLI/LSP:** The live command path exists, but `/trace/query` currently returns a JSON status of unavailable with the reason that live trace HTTP query execution is not wired to the backend actor yet. **Recommendation:** Treat fe dev trace live as discovery scaffolding, not a working live query surface. Wire the endpoint next.

- **[High] Real trace causality:** Real trace emission currently gives MIR/runtime facts and bytecode opcode/gas inventory, but not enough to explain loop costs, stack residency, or zext causality from compiler phases. **Recommendation:** Preserve the current honest unavailable behavior and implement causal facts phase by phase.
- **[High] Owner keys:** Runtime owner keys like `runtime:{function.symbol}` and codegen owner keys like `contract:{contract_name}:runtime` are probably too weak for packages, modules, duplicate names, generics, or future inline instances. **Recommendation:** Introduce stable package/module/function/runtime-instance owner keys before traces become durable artifacts.
- **[Medium-High] Trace schema validation:** `OriginNodeFact` stores a kind next to an `OriginExportKey` that already carries kind; that duplicates identity metadata unless validated. Some validating newtypes derive transparent `Deserialize`, which may bypass constructors. **Recommendation:** Either remove redundant kind fields or validate equality with `key.kind`. Add `serde try_from` for validating newtypes and expand validator coverage.
- **[Medium-High] Static gas semantics:** The gas demo is static opcode-table inventory, not execution gas tracing. Dynamic opcodes and storage behavior cannot be treated as precise runtime cost. **Recommendation:** Use labels like static opcode gas inventory. Show confidence per opcode and avoid overconfident totals where dynamic gas dominates.
- **[Medium] Inlay anchoring and cost:** Current trace inlays attach at the request range start and build the service per request. This is fine as a demonstration but not the final UX. **Recommendation:** Cache by document version/config hash and anchor hints only when facts provide real spans or loop/function regions.
- **[Medium] Module size / layering:** `crates/fe/src/trace.rs` is now about 935 lines and mixes CLI dispatch, fixture construction, real emit, validation, live HTTP, rendering, and tests. **Recommendation:** Split into `trace_emit`, `trace_fixture`, `trace_render`, `trace_live`, and `trace_cli` glue modules.
- **[Medium] Legacy fact overlap:** `common::facts` and `TypedFactSet` still exist for `analyze/legacy` paths, while `trace-facts` is the new substrate. **Recommendation:** Keep `common::facts` quarantined as `legacy/analyze-only` and migrate remaining uses when practical.
- **[Medium] HTTP auth:** `.fe-lsp.json` records auth metadata and `token_file`, but `/trace/query` is currently a stub and does not appear to enforce token-based access. **Recommendation:** Before returning live trace data over HTTP, add localhost binding checks and token validation.

5. Module-by-module review

README.md

Strong improvement. It explicitly distinguishes `fe dev trace-fixture` from `fe dev trace` and states the current gaps. This protects trust and should remain the public wording until real causal trace facts exist.

[crates/trace-facts/src/fact.rs](#)

Good compact model. Opcode and GasCost are useful additions. The main cleanup is schema rigor: validating deserialization, reducing duplicate kind metadata, and keeping relation_name as a view helper rather than a schema authority.

[crates/trace-facts/src/jsonl.rs](#)

Good boundary artifact. TraceMetadata with schema_version, data_source, target, command, input_path, flags, and fixture_marker is exactly the kind of evidence reports should carry. Add schema-version compatibility checks when reading bundles.

[crates/trace-facts/src/validate.rs](#)

Much better than before. It checks duplicate nodes, instruction identity/site duplication, missing endpoints, bad storage phase/location pairs, empty mnemonics, category-without-instruction, loop membership integrity, inline endpoints, opcode text, and gas subjects. Next validation should check node.kind vs key.kind, duplicate/ambiguous categories, opcode subject kind, schedule strings, metadata strings, and warning use for unknown categories.

[crates/trace-facts/src/snapshot.rs](#)

Good idea to validate before building a snapshot. The fnv64 trace hash is acceptable as a debug fingerprint, but should be documented as non-cryptographic and replaced with sha256 if used for cache trust or external artifact identity.

[crates/trace-query/src/lib.rs](#)

Good read-only service boundary. loop_cost is honest when loop membership is absent. gas_breakdown correctly works over GasCost facts. The local-name lookup currently depends on local_key display text; add canonical SymbolFact/DisplayNameFact before relying on it for real programs.

[crates/mir/src/trace.rs](#)

Good phase-owned start. MIR emits runtime local/statement/terminator origin nodes and MIR storage facts. The storage reason is still Unknown, and there are no HIR/source origin edges yet. Strengthen owner keys and add source/local labels next.

[crates/codegen/src/trace.rs](#)

Useful actual bytecode pass. It emits bytecode function, PC/instruction, category, opcode, and static gas facts from emitted EVM bytecode. Keep it explicitly codegen-owned. Next steps are origin edges to Sonatina/MIR when available and better dynamic-gas confidence handling.

[crates/fe/src/trace.rs](#)

Functionally useful but too broad. It now includes real trace emit, fixture emit, renderers, live HTTP client, query commands, and tests. Split before adding more features. The tests that assert real trace is compiler_emitted and loop-cost remains unavailable are valuable.

[crates/language-server/src/introspection.rs](#)

This is the right LSP-side bridge: build facts from the live database, create TraceBundle metadata, validate into TraceSnapshot, and expose TraceIntrospectionService. It should become the implementation behind /trace/query.

[crates/language-server/src/combined_server.rs](#)

HTTP docs/config endpoints and /lsp WebSocket are useful. /trace/query is explicitly stubbed. Next push should make this endpoint real or keep live CLI commands hidden behind an experimental flag.

[crates/language-server/src/functionality/inlay_hints.rs](#)

Config-gated integration is a good start. Inlays should remain low-noise. Current hints use the request range start as a temporary anchor; the polished version needs span-backed anchoring and caching.

[crates/introspection-config/src/lib.rs](#)

The config surface is broad but coherent: LSP enablement, trace mode, max analysis time, max hints per file, inlay hint modes, hover, code lens, trace, gas, and live options. Defaults are appropriately low-noise, with gas and loop-cost inlays off by default.

6. Sean/Fibonacci probing status

The branch now has two distinct Fibonacci stories, and keeping them separate is the key to trust.

- **Fixture path:** fe dev trace-fixture emits a canned RISC-V-like loop and can demonstrate the desired loop-cost/explain-local UX: b stack traffic, repeated zexts, n hoist candidate, and scalar promotion guidance. This is useful as a UX contract.
- **Real compiler-emitted path:** fe dev trace emit produces compiler_emitted trace bundles with MIR facts and actual EVM bytecode instruction/opcode/static gas facts. It cannot yet derive Sean-style causal answers about why b is stack-resident or where zexts came from.
- **Current honest behavior:** For real traces, loop-cost says loop cost is unavailable and presents an available compiler-derived bytecode summary. This is the right trust-preserving behavior.
- **Next causal requirements:** To make the real trace answer Sean-style questions, add loop membership and loop source spans, source-local labels, HIR-to-MIR origin edges, MIR-to-codegen/bytecode edges, storage reasons, backend spill/reload facts, integer legalization events, and value-property facts such as known zero-extension and loop invariance.

7. LSP/CLI live connection assessment

The live connection architecture is close to the shape we wanted, but it is not wired through yet.

Current live flow:

fe lsp writes .fe-lsp.json with PID, endpoint URLs, capabilities, config hash, and auth metadata

fe dev trace live reads .fe-lsp.json

origin-overhaul-phased latest push review - 2026-05-24

```
fe dev trace live POSTs JSON to <trace_api_url>/query
language-server /trace/query returns status = unavailable
```

Target live flow:

```
fe dev trace live POSTs a typed request containing uri/path, query kind,
schedule/local/loop params, config hash, and auth token
combined_server authenticates and forwards to the backend actor
backend builds or retrieves TraceIntrospectionService for that file/document version
trace-query produces a typed JSON report
CLI renders human output or --json output
LSP inlays/hovers/code lenses consume the same report path
```

This should be implemented before adding more live UX surface. Otherwise the CLI and LSP will drift: CLI will read JSONL snapshots while LSP inlays directly call `service_for_file`, and the HTTP path will remain a misleading placeholder.

8. Gas-cost demo assessment

The gas-cost direction is promising, especially for smart-contract developer UX, but the first version should be named carefully.

Recommended language: "static opcode gas inventory" or "static bytecode gas summary". Avoid "gas tracing" unless runtime execution traces are actually collected and joined back to bytecode/source origins.

Good first LSP UX: a disabled-by-default summary inlay at the contract/function level, a hover or command that opens a bytecode static gas breakdown, and a CLI command that can output JSON for agents. Bad first LSP UX: putting a whole-file static total at an arbitrary cursor range without a source span or confidence caveat.

What to show	Best surface	Caveat
Whole-artifact static opcode total	Command output / code lens / hover summary	Not runtime gas; may include dynamic opcode underestimates
Top static opcode contributors	Hover or panel	Sort by static cost but include confidence
Opcode rows	CLI JSON / browser table	Useful for agents, too detailed for inlays
Source-attributed gas	Future inlay/hovers	Requires bytecode-to-source origin edges
Runtime gas trace	Future debugger/trace view	Requires execution trace plus source mapping

9. SSOT and schema guardrails

- Compiler phases own identity and decisions. Trace-facts serializes them. Trace-query derives reports. Reports never define truth.

- Every fact that mentions a compiler entity must use `OriginExportKey` or a typed wrapper around it; no stringly local names as identity.
- If `OriginNodeFact` keeps a `kind` field, validate it against `OriginExportKey.kind`. Prefer not to duplicate identity metadata unless it adds real display value.
- Do not hand-maintain relation schemas. If relation tables return later, generate them from typed facts or keep them entirely downstream.
- Keep fixture facts and compiler-emitted facts separated by metadata and command names.
- Require `TraceValidator` before any report, inlay, hover, HTTP response, or agent query trusts a trace.
- Base facts are emitted by compiler phases. Derived facts are named and versioned by query rules. Datalog, if added, must be read-only and downstream.

10. Recommended next phased push

Phase	Work	Acceptance
Phase 0: Freeze and verify	Run and record cargo check/test for the touched crates. Keep zext-report gated. Preserve README wording that real traces have explicit gaps.	Published test matrix; no new broad UX surface; failures fixed before feature work.
Phase 1: Harden trace schema	Enforce <code>schema_version</code> compatibility; add validating <code>Deserialize</code> for <code>OriginNodeKind</code> , <code>CompilerReason</code> , <code>EvmSchedule</code> , and similar text wrappers; validate <code>node.kind == key.kind</code> or remove <code>node.kind</code> ; validate duplicate categories and metadata strings.	Malformed JSONL cannot create invalid typed facts; validator catches duplicated or contradictory trace records.
Phase 2: Split trace CLI module	Move fixture construction, real emit, renderers, live client, and command dispatch into separate modules under <code>crates/fe/src/trace/*.rs</code> or an equivalent module tree.	No behavior change; smaller reviewable files; tests moved with owning modules.
Phase 3: Strengthen owner/export keys	Replace runtime: <code>{function.symbol}</code> and contract: <code>{contract_name}:runtime</code> with <code>package/module/function/runtime-instance</code> keys that cannot collide across packages, modules, overload-like forms,	Synthetic collision tests pass; same function name in different modules produces different owner keys.

	generic instances, or future inlining.	
Phase 4: Wire /trace/query	Define typed HTTP requests/responses; require uri/path in live requests; validate auth token; call language-server introspection service; return loop-cost/explain-local/gas-breakdown reports as JSON.	fe dev trace live gas-breakdown returns a real report from a running LSP, not the unavailable stub.
Phase 5: Polish LSP config and hints	Cache introspection by document version/config hash; keep gas/loop inlays off by default; anchor gas at contract/function only; show loop hints only when loop facts have spans; use hovers for detail.	No noisy hints by default; enabled hints are stable, cached, and confidence-labeled.
Phase 6: Add causal compiler facts	Add Symbol/DisplayName facts, HIR-to-MIR origin edges, MIR-to-bytecode/codegen edges where available, storage reasons, loop membership/source spans, integer legalization events, and value-property facts.	Real trace reports can explain at least one local storage story and one compiler event without fixture data.
Phase 7: Convert Sean/Fib from fixture to real trace	Use the fixture as expected UX, but drive the report from compiler_emitted facts. The fixture remains a regression demo; the real trace becomes the engineering signal.	For fib_demo.fe, real trace can say why loop-cost is available, where local storage is determined, and which zext/legalization events exist or are absent.
Phase 8: Optional Datalog layer	Only after base facts are stable, add a read-only derived query layer. Keep base vs derived facts separated and version derived rulesets.	Datalog can answer multi-hop questions without becoming the SSOT.

11. Acceptance criteria for the next push

1. fe dev trace emit fib_demo.fe writes compiler_emitted JSONL with exactly one metadata record and passes TraceValidator with no errors.
2. fe dev trace validate rejects malformed metadata, duplicate metadata, mismatched node kind/key kind, duplicate instruction sites, empty opcode text, empty classifier versions, and bad storage phase/location pairs.

3. `fe dev trace gas-breakdown --from target/fib.trace.jsonl` reports a static opcode inventory with schedule and confidence labels, not runtime gas tracing language.
4. `fe dev trace loop-cost` over a real trace remains unavailable until loop facts exist, and explicitly says what facts are missing.
5. `fe dev trace live gas-breakdown` returns a real JSON response from the LSP HTTP server and validates an auth token or explicit local-only security policy.
6. LSP gas inlays are disabled by default, enabled by config, capped by `max_hints_per_file`, cached, and confidence-labeled.
7. No new relation-table or report schema duplicates typed facts by hand. Any view is generated or recomputed from TraceFact records.
8. README and CLI help distinguish fixture reports, compiler-emitted traces, static gas inventory, and future runtime gas tracing.

12. Implementor directive

Do not add another broad feature layer yet. The next push should make the existing trace/LSP/gas scaffolding trustworthy and wired.

Priority order:

1. Harden the trace schema and validator.
2. Split the trace CLI module without behavior changes.
3. Strengthen owner/export keys.
4. Wire `/trace/query` to the existing LSP introspection service with auth.
5. Polish config-driven inlays and caching.
6. Add the first real causal facts needed to replace the Fibonacci fixture with compiler-derived answers.

Forbidden for now:

- No universal `OriginNode` enum in common.
- No new relation-table system.
- No Datalog until base facts are stable.
- No runtime gas-tracing claims from static opcode facts.
- No LSP hint that cannot explain its data source and confidence.

Appendix A. Reviewed source map

Area	URL
Branch README	https://github.com/micahscopes/fe/tree/origin-overhaul-phased
Developer trace README section	https://github.com/micahscopes/fe/tree/origin-overhaul-phased#developer-trace-prototype
CLI trace command implementation	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/fe/src/trace.rs
CLI main command wiring	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/fe/src/main.rs
Trace fact model	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-facts/src/fact.rs
Trace JSONL bundle model	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-bundle/src/fact.rs

	overhaul-phased/crates/trace-facts/src/jsonl.rs
Trace snapshot validation boundary	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-facts/src/snapshot.rs
Trace validator	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-facts/src/validate.rs
Trace query service	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-query/src/lib.rs
MIR trace emitter	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/mir/src/trace.rs
MIR origin types	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/mir/src/origin.rs
Codegen trace emitter	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/codegen/src/trace.rs
Language-server combined HTTP/WS server	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/language-server/src/combined_server.rs
Language-server introspection bridge	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/language-server/src/introspection.rs
LSP inlay hints	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/language-server/src/functionality/inlay_hints.rs
LSP discovery metadata	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/fe/src/doc.rs
Introspection configuration	https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/introspection-config/src/lib.rs